Secure Design Document

Secure Gallery Access Tracker

Build It; Break It; Fix It (BIBIFI) - Group 6

Secure Programming
Matthew Wear | Brandon Renner
Design baseline: Phase I requirements and threat model

Executive Summary

The Secure Gallery Access Tracker is a browser-based application for recording and monitoring the movement of guests and employees through an art gallery. Authorized Employees and Administrators record gallery and room entry/exit events, view current occupancy, and search historical records. Administrators additionally manage users, roles, rooms, and audit information.

The design uses Python and Flask for the backend, HTML/CSS with Jinja for the user interface, SQLAlchemy for database access, SQLite for persistence, and Flask session-based authentication. Security is enforced in the backend through authentication, role-based access control, server-side validation, parameterized database operations, protected session handling, business-state validation, and audit logging.

Table of Contents

- 1. System Overview
- 2. Architecture and Trust Boundaries
- 3. Users, Roles, and Authorization
- 4. Functional Requirements
- 5. Security Requirements
- 6. Data-Flow Model
- 7. Data Model and ERD
- 8. STRIDE Threat Analysis
- 9. Security Control Design
- 10. Verification Strategy
- 11. Open Decisions and Missing Information
- Appendix A. Requirement Traceability

1. System Overview

1.1 Purpose

The web-based application service manages and monitors access within an art gallery. It tracks persons entering and leaving the gallery, entering and leaving individual rooms, current occupancy, and historical movement events. It also provides account, role, room, and audit administration.

1.2 Scope

- User authentication and logout
- Role-based access control
- Gallery entry and exit tracking
- Room entry and exit tracking
- Current gallery and room occupancy
- Historical event search
- Administrative user, role, and room management
- Security and audit logging

1.3 Technology Stack

Component	Technology
Backend	Python / Flask
Frontend	HTML, CSS, Jinja
Database	SQLite
Database access	SQLAlchemy
Authentication	Flask session-based authentication
Client platform	Web browser

1.4 Key Design Assumptions

- Application users and tracked persons are separate concepts. A tracked person does not require an application account.
- All authorization decisions are made by the Flask backend, not by hidden interface elements.
- GalleryEvent records form the authoritative movement history. Current state is derived from valid event sequences
- The deployment may be local for coursework, but HTTPS is required whenever traffic crosses an untrusted network.
- SQLite is acceptable for the project scale, provided transaction handling and concurrency limitations are explicitly tested.

2. Architecture and Trust Boundaries

2.1 Logical Architecture

Layer	Responsibilities	Primary controls
Browser / UI	Render Jinja pages; submit login, event, search, and administration requests	Output encoding, secure cookies, CSRF defense, no client-side authorization trust
Flask routes	Receive requests and produce responses	Authentication, endpoint authorization, input limits, generic errors
Application services	Authorization, event-state validation, administration, audit generation	Least privilege, allowlisted fields, transaction boundaries
SQLAlchemy data access	Read and write persistent records	Parameterized queries, ORM binding, rollback
SQLite database	Persist users, roles, persons, rooms, events, and audit logs	Constraints, file permissions, backups, integrity checks

2.2 Trust Boundaries

TB-1 Browser to Flask: All browser-controlled values are untrusted. Authentication, server-side validation, request limits, and authorization apply at this boundary.

TB-2 Flask routes to protected services: Route handlers may only invoke protected business operations after identity and role checks. Internal service functions must not assume the caller is authorized.

TB-3 Application to SQLite: All database access uses parameter binding or ORM operations. Transactions, constraints, and rollback preserve data integrity.

TB-4 User privilege boundary: Guest/User, Employee, and Administrator capabilities are separated. Direct endpoint requests must be checked even when the UI hides unavailable functions.

TB-5 Deployment and secrets boundary: The Flask secret key and other sensitive settings remain outside source control and are available only to the deployed application process.

3. Users, Roles, and Authorization

3.1 Role Definitions

Role	Description
Guest/User	Authenticated user with limited access to information associated with the user account.
Employee	Gallery staff permitted to record and monitor movement and search authorized history.
Administrator	Privileged staff permitted to manage users, roles, rooms, and audit information.

3.2 Permission Matrix

Function	Guest	Employee	Admin
Login / logout	Yes	Yes	Yes
View own account and status	Yes	Yes	Yes
View gallery / room occupancy	**No**	Yes	Yes
Record gallery / room event	**No**	Yes	Yes
Search event history	**No**	Yes	Yes
Manage users	**No**	**No**	Yes
Assign roles	**No**	**No**	Yes
Manage rooms	**No**	**No**	Yes
View audit logs	**No**	**No**	Yes

4. Functional Requirements

ID	Requirement	Description
FR-01	User Authentication	Allow registered users to log in with a username and password.
FR-02	User Logout	Allow authenticated users to log out.
FR-03	Role Assignment	Associate each application user with Guest/User, Employee, or Administrator.
FR-04	Account Information	Allow authenticated users to view their own account information.
FR-05	Gallery Entry Recording	Allow Employees and Administrators to record a person entering the gallery.
FR-06	Gallery Exit Recording	Allow Employees and Administrators to record a person leaving the gallery.
FR-07	Room Entry Recording	Allow Employees and Administrators to record a person entering a room.
FR-08	Room Exit Recording	Allow Employees and Administrators to record a person leaving a room.
FR-09	Current Gallery Occupancy	Allow Employees and Administrators to view the current number of persons in the gallery.
FR-10	Current Room Occupancy	Allow Employees and Administrators to view occupants of each room.
FR-11	Historical Event Search	Allow Employees and Administrators to search/view movement history.
FR-12	User Management	Allow Administrators to create, modify, disable, and manage accounts.
FR-13	Role Management	Allow Administrators to assign or modify roles.
FR-14	Room Management	Allow Administrators to create, modify, and manage rooms.
FR-15	Audit Log Access	Allow Administrators to view application audit logs.

5. Security Requirements

ID	Requirement	Description
SR-01	Secure Authentication	The system should authenticate users with a valid username and password before granting access to protected functionality
SR-02	Secure Password Storage	The system should store passwords using a secure salted password-hashing algorithm. Plaintext passwords should never be stored, logged, or committed to the repository
SR-03	Role-Based Access Control	The system should enforce Guest/User, Employee, and Administrator permissions on the server side for every protected operation
SR-04	Least Privilege	The system should grant only the functionality and information required by the assigned role and deny access by default when permission is not explicitly granted
SR-05	Secure Session Management	The system should securely create, maintain, expire, and invalidate sessions. Cookies should use appropriate security attributes, logouts should prevent reuse of the ended session.
SR-06	Input Validation	The system should validate all user-controlled input on the server for expected type, format, length, range, and allowed values before processing or storage
SR-07	Injection Prevention	The system should use parameterized queries, SQLAlchemy binding, or equivalent safe mechanisms for database access. Input should not be concatenated into SQL commands.
SR-08	Cross-Site Scripting Prevention	The system should encode or escape untrusted content before HTML rendering and prevent stored and reflected XSS.
SR-09	Business-Logic Integrity	System should validate gallery/room events before recording and reject invalid, conflicting, or out-of-order transitions.
SR-10	Data Integrity & Transaction Safety	The system should process security-sensitive database operations atomically and roll back the operation if authorization, validation, or persistence fails.
SR-11	Audit and Security Logging + Protection	The system should record authentication, movement, administration, access-control failures, and other security-relevant activities. Administrators may view audit logs; audit records should be protected from unauthorized alteration/deletion.
SR-12	Information-Disclosure Prevention	The system should prevent unauthorized disclosure of account data, movement history, occupancy, audit records, database details, configuration, and other protected information.
SR-13	Secure Error Handling	System should return generic errors to users while recording appropriate diagnostic information in protected server logs.
SR-14	Secrets and Secure Configuration Management	The system should keep secret keys, credentials, and sensitive configuration outside the source-code repository.
SR-15	Availability and Resource Protection	System should limit authentication attempts, request sizes, search ranges, query results, and other resource-intensive operations.
SR-16	Transport Security	The system should use HTTPS when deployed across an untrusted network to protect credentials, session cookies, and application data in transit.

6. Data-Flow Diagram

The data-flow model shows the external user roles, internal Flask processes, persistent data stores, and trust boundaries. Requests cross the browser trust boundary before authentication, authorization, validation, business processing, database access, and audit recording.

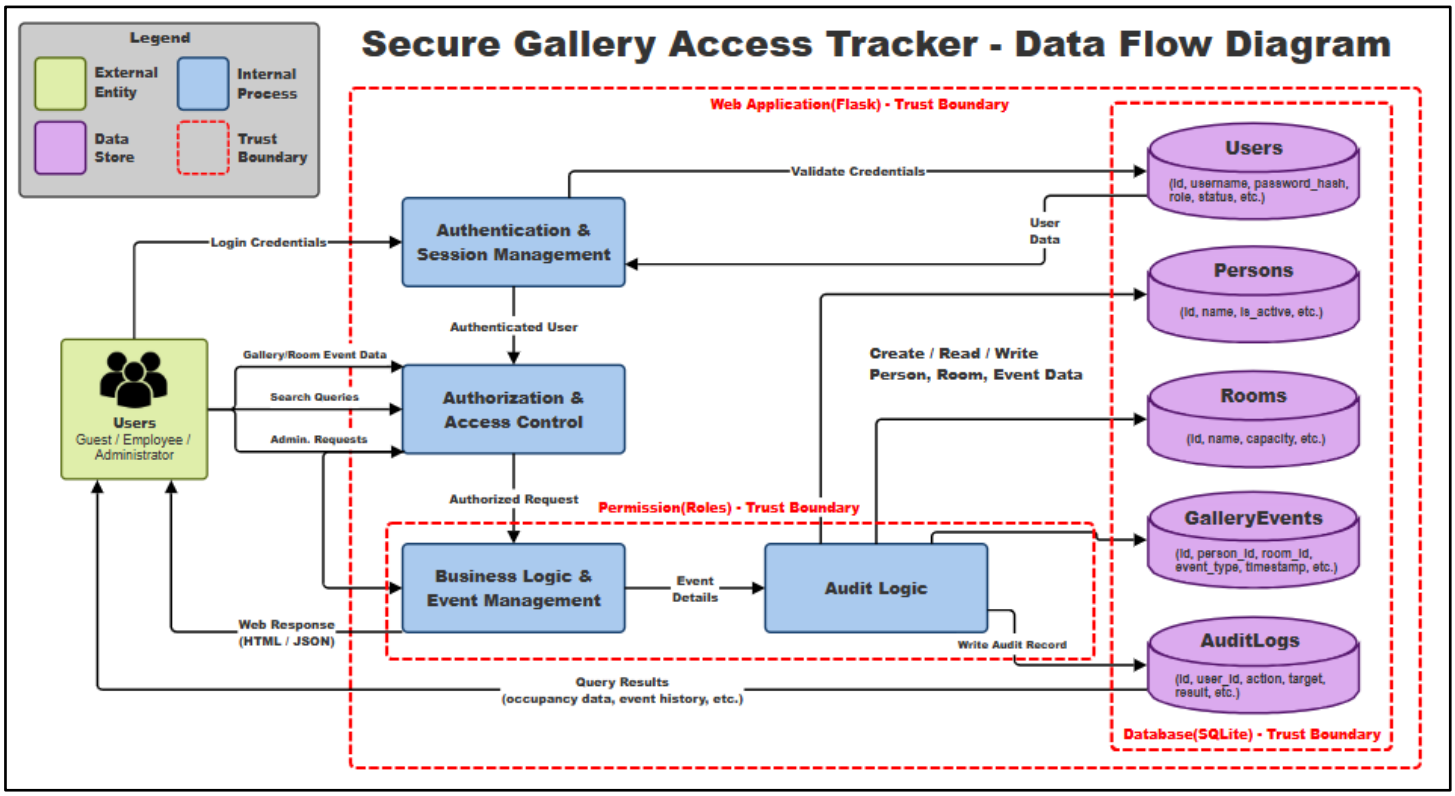


Figure 1. Data-flow diagram for the Secure Gallery Access Tracker

6.1 Principal Data Flows

- Login credentials → Authentication and Session Management
- Authenticated identity and role → Authorization and Access Control
- Gallery/room movement data → Business Logic and Event Management
- Search criteria → Event-history and occupancy processing
- Administrative requests → Protected user, role, and room operations
- Validated database operations → Users, Persons, Rooms, GalleryEvents, and AuditLogs
- Security-relevant results → Audit Logic → Audit Logs
- Authorized HTML/status/query results → Browser

7. Data Model and ERD

The data model separates application identities from tracked persons. GalleryEvents connect a tracked Person, an optional Room, and the User who recorded the event. AuditLogs associate security-relevant actions with an acting User when an authenticated actor exists.

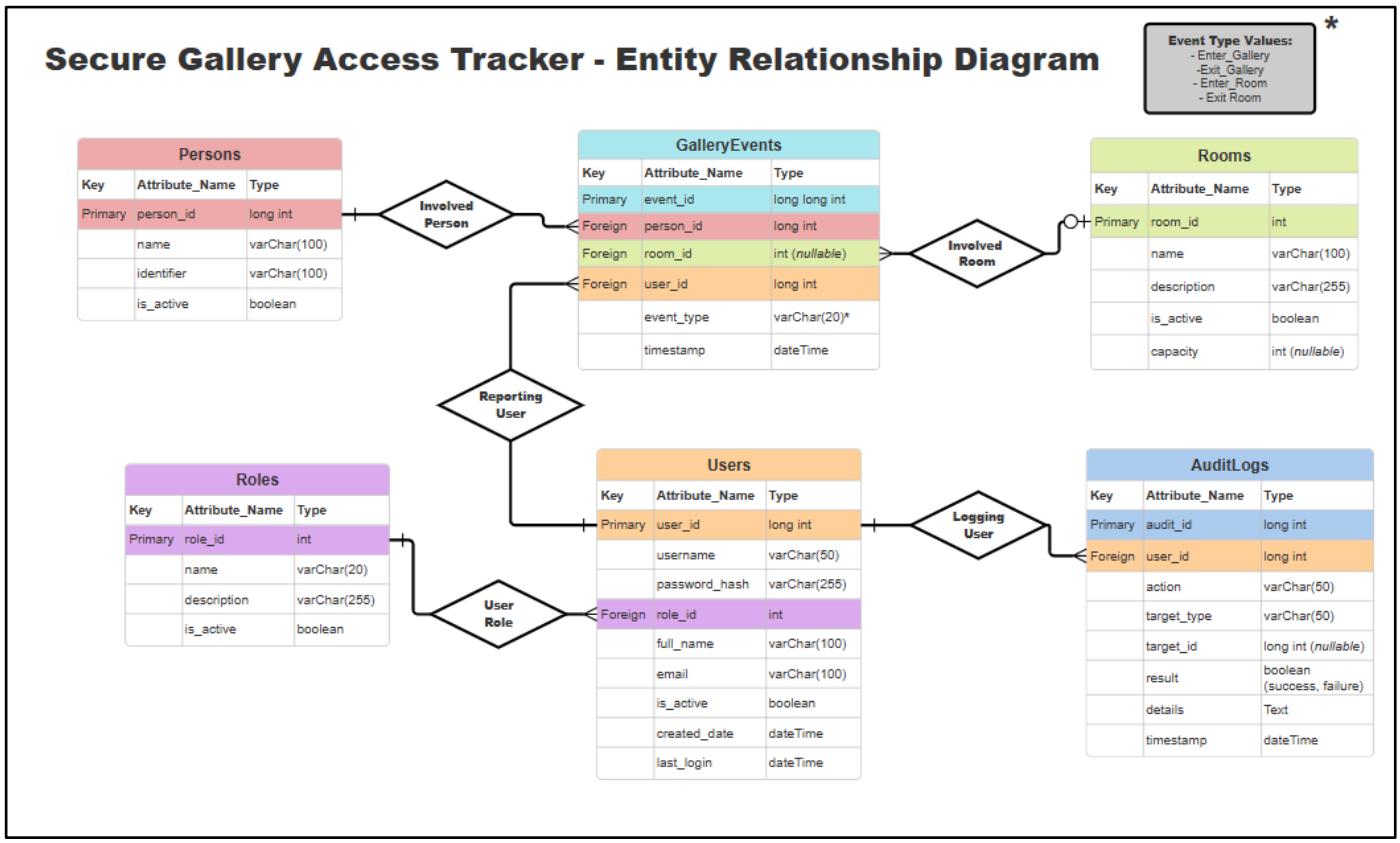


Figure 2. Entity-relationship diagram for users, roles, persons, rooms, gallery events, and audit logs.

7.1 Entity Responsibilities and Constraints

Entity	Purpose / key constraint
Roles	Defines Guest/User, Employee, and Administrator roles. Role names are unique.
Users	Stores username, password hash, role, and activation state. Username is unique; passwords are never stored directly.
Persons	Represents individuals whose physical location is tracked. A Person may exist without a User account.
Rooms	Represents gallery rooms. Inactive rooms remain for historical integrity.
GalleryEvents	Stores ENTER_GALLERY, EXIT_GALLERY, ENTER_ROOM, and EXIT_ROOM events. Room is required only for room events.
AuditLogs	Stores security-relevant actions, target, result, details, and timestamp. Application functionality does not edit or delete existing audit records.

8. STRIDE Threat Analysis

S-01 - Credential-based user spoofing

STRIDE category	Spoofing
Affected components	Login endpoint, user database, authentication service
Potential impact	Unauthorized account or privileged access; fraudulent events or administration
Design mitigation	Salted password hashing, generic login errors, failed-login monitoring, login-attempt limits, server-side role enforcement
Related requirements	SR-01, SR-02, SR-03, SR-11, SR-15

S-02 - Session hijacking or fixation

STRIDE category	Spoofing
Affected components	Flask session cookie, browser connection, logout
Potential impact	Impersonation without credentials; continued access after logout
Design mitigation	Secure cookies, session expiration and invalidation, session renewal, HTTPS on untrusted networks
Related requirements	SR-05, SR-14, SR-16

T-01 - Modified movement request

STRIDE category	Tampering
Affected components	Gallery and room event endpoints
Potential impact	False occupancy, incorrect history, invalid state
Design mitigation	Authorize actor, validate IDs and event type, use server timestamps, enforce state transitions
Related requirements	SR-03, SR-04, SR-06, SR-09, SR-10

T-02 - SQL injection through search or administration

STRIDE category	Tampering
Affected components	Search, login, user and room management
Potential impact	Authentication bypass, disclosure, modification, deletion, database corruption
Design mitigation	SQLAlchemy binding/parameterized queries, allowlisted fields, generic errors
Related requirements	SR-06, SR-07, SR-08, SR-11

R-01 - Denial of movement or administrative action

STRIDE category	Repudiation
Affected components	Event service, administrative endpoints, audit logic
Potential impact	Inability to attribute or reconstruct actions

STRIDE category	Repudiation
Design mitigation	Record actor, action, target, result, and server timestamp
Related requirements	SR-01, SR-03, SR-09

R-02 - Audit-log deletion or manipulation

STRIDE category	Repudiation
Affected components	AuditLogs, admin audit route, SQLite file
Potential impact	Lost evidence and concealed changes
Design mitigation	Append-only application behavior, admin read access only, database file protection
Related requirements	SR-10, SR-11, SR-13

I-01 - Stored or reflected XSS

STRIDE category	Information Disclosure
Affected components	Rendered names, rooms, searches, event history, errors
Potential impact	Disclosure or privileged browser actions
Design mitigation	Jinja escaping, context-safe output, input limits, secure cookie attributes
Related requirements	SR-05, SR-06, SR-07, SR-08, SR-12

I-02 - IDOR or unauthorized record access

STRIDE category	Information Disclosure
Affected components	Account, person, history, occupancy, audit and admin routes
Potential impact	Exposure or modification outside assigned permissions
Design mitigation	Object-level server authorization; never equate identifier knowledge with permission
Related requirements	SR-02, SR-03, SR-04, SR-12

D-01 - Login or search resource exhaustion

STRIDE category	Denial of Service
Affected components	Login, historical search, occupancy, Flask, SQLite
Potential impact	Slow or unavailable service; delayed event recording
Design mitigation	Login rate limits, request limits, bounded date ranges, pagination/result limits
Related requirements	SR-05, SR-06, SR-07, SR-15

D-02 - Concurrent conflicting state transitions

STRIDE category	Denial of Service
Affected components	Event service and database transactions
Potential impact	Invalid occupancy, database locks, unavailable event recording

STRIDE category	Denial of Service
Design mitigation	Validate and write in one transaction; reject conflicts; rollback on failure
Related requirements	SR-09, SR-10, SR-13

E-01 - Role or endpoint authorization bypass

STRIDE category	Elevation of Privilege
Affected components	Route decorators, authorization service, admin endpoints
Potential impact	Unauthorized administration or audit access
Design mitigation	Authenticate and authorize every protected backend endpoint; deny by default
Related requirements	SR-01, SR-03, SR-04, SR-05, SR-10

E-02 - Unauthorized role assignment or mass assignment

STRIDE category	Elevation of Privilege
Affected components	User creation and modification
Potential impact	Ordinary user obtains Administrator access
Design mitigation	Allowlist writable fields; separate profile and role operations; audit attempts
Related requirements	SR-03, SR-04, SR-05, SR-06, SR-07, SR-11

9. Security Control Design

Control area	Design approach
Authentication	Password hashes; generic login response; reject inactive accounts; login attempts bounded
Authorization	Reusable backend checks for logged-in state, role, and requested object; default deny
Sessions	Strong Flask secret; Secure, HttpOnly, and SameSite cookie settings; idle/absolute expiration; invalidate on logout; renew after login or privilege change.
Input and output	Server-side type, range, length, and allowlist validation; Jinja autoescaping; no unsafe rendering of user content.
Database	SQLAlchemy binding or parameterized SQL; foreign keys and check constraints where supported; transaction rollback on error.
Movement-state integrity	Check current state before each event; reject duplicate entry, exit while absent, room entry while outside, wrong-room exit, and multiple-room occupancy.
Audit	Record actor, action, target, outcome, and timestamp; do not record passwords, session values, secret keys, or CSRF tokens; no edit/delete route.
Errors	Generic client response; detailed errors only in protected logs; debug disabled outside dev.
Configuration	Secrets loaded from environment or protected local configuration; .env and local database artifacts excluded from Git.
Availability	Rate limits, body-size limits, paginated and bounded searches, safe timeouts, and controlled handling of SQLite lock errors.
Transport	HTTPS when requests cross an untrusted network; no credentials/session identifiers in URL.

10. Verification Strategy

The following tests should become executable evidence for Build It, Break It, and Fix It. The exact framework and thresholds should be documented once selected.

- **Authentication**: valid login, invalid password, unknown username, disabled account, logout, expired session, reused session.
- **Authorization**: direct requests to every protected route as Guest/User, Employee, and Administrator.
- Injection: SQL metacharacters in login, search, person, room, user, and event fields.
- **XSS**: harmless stored and reflected payloads in every rendered text field.
- **IDOR**: substitute account, person, room, event, and audit identifiers.
- **Business Logic**: duplicate gallery entry, exit while absent, room entry while outside, entry into two rooms, wrong-room exit, and gallery exit while still in a room.
- **Concurrency**: simultaneous conflicting events for the same person.
- **Error Disclosure**: malformed requests and forced database errors must not reveal stack traces, SQL, paths, or secrets.
- **Availability**: verify login limits, maximum body size, pagination, date-range bounds, and maximum result size.
- **Regression**: create a test for every confirmed vulnerability before closing the Fix It item.

Appendix A. Requirement Traceability

Requirement	Primary threats	Verification evidence
SR-01	S-01, R-01, E-01	Login and protected-route authentication tests
SR-02	S-01, I-01	Database inspection; password verification tests
SR-03	T-01, I-02, E-01, E-02	All routes tested under all roles
SR-04	T-01, I-02, E-01, E-02	Default-deny and object-access tests
SR-05	S-02, I-01	Cookie, expiration, logout, and reuse tests
SR-06	T-01, T-02, I-01, D-01, E-02	Boundary, type, length, and unexpected-field tests
SR-07	T-02	SQL injection tests and code review
SR-08	I-01	Stored and reflected XSS tests
SR-09	T-01, R-01, D-02	Invalid and conflicting state-transition tests
SR-10	T-01, R-02, D-02	Rollback and concurrency tests
SR-11	S-01, R-01, R-02, E-02	Audit content and access-control tests
SR-12	I-01, I-02	Unauthorized data-access tests
SR-13	T-02	Error-disclosure tests
SR-14	S-02, R-02	Repository/configuration review
SR-15	D-01, D-02	Rate, size, range, and result-limit tests
SR-16	S-02	Deployment/TLS verification